SUPPLYSENSE

# Project overview and class presentation guide

A full-stack retail, inventory and replenishment platform

This guide explains the project’s purpose, users, architecture, main workflows, demand intelligence, security, technology choices, current status and a suggested live demo. It is written as a speaking aid for a classroom presentation.

Project name: SupplySense
Project type: Full-stack web application
Primary goal: connect customer orders, warehouse stock, suppliers and replenishment decisions.

# 1. Project at a glance

SupplySense is a multi-warehouse retail and replenishment platform. It connects customer commerce to warehouse operations and supplier purchasing. Sales and stock movements become operational data that managers can use to understand demand and decide what to reorder.

### Problem being addressed

Retail stock is often split across warehouses while sales, supplier purchasing and replenishment are managed separately. This can lead to stockouts, overselling, delayed reorders and limited visibility into why a purchase is recommended.

### Project goals

Sell products without overselling; keep stock accurate for every SKU and warehouse; record auditable stock movements; support supplier replenishment through receipt; provide explainable demand and reorder information; and enforce role and warehouse boundaries.

### Scope

The first release includes authentication, roles, product catalog, warehouses, warehouse-specific inventory, cart and wishlist, reservations, orders, mock payments, suppliers, purchase orders, shipments, goods receipt, traceability, dashboards, audit history and demand intelligence.

# 2. Users and what they do

| Role | Main responsibility |
|---|---|
| Customer | Browse products, manage cart, place orders and track them. |
| Warehouse manager | Manage assigned stock, receipts, purchase orders and recommendations. |
| Supplier | Maintain supplied products and respond to assigned purchase orders and shipments. |
| Administrator | Manage platform configuration, users, products, warehouses and suppliers. |
| Auditor | Read operational and security history without changing business data. |

# 3. Architecture and data flow

The browser runs a React single-page app built with Vite. It calls a REST API built with Node.js and Express. The API is organized as a modular monolith: features such as catalog, inventory, orders, suppliers, intelligence and audit have clear service boundaries but run in one backend application.

**Request flow:** React client → request logging and security middleware → authentication and role/scope checks → route/controller → business service → Prisma data layer → PostgreSQL transaction → response.

PostgreSQL is the source of truth for users, products, inventory, orders, payments, suppliers, forecasts, recommendations and audit records. Redis is optional and may hold disposable cache data only; it is not

authoritative for stock. Python demand prediction is launched by the backend as a subprocess when available, rather than running as a separate service.

### Why these choices

A modular monolith keeps deployment and transactional workflows understandable for the first release. PostgreSQL supports relational integrity and transactions needed for inventory and checkout. Prisma centralizes schema access and migrations. Interfaces keep the intelligence and payment implementations replaceable.

## 4. Main workflows

### Customer purchase

The customer selects a SKU and checks out. The server recalculates the price and validates product state, address and stock. Inside a PostgreSQL transaction, the system reserves all order lines and creates a pending-payment order. An idempotency key prevents repeating the same checkout from creating duplicates. A successful mock payment confirms the order; failure releases the reservation.

### Supplier replenishment

A manager reviews a reorder recommendation and creates a draft or submitted purchase order according to the workflow. The supplier can see only assigned purchase orders, then accept or reject and record shipment information. A warehouse manager records full or partial goods receipt. Receipt updates warehouse inventory and the purchase-order/shipment state.

### Inventory traceability

Stock is recorded separately for each SKU and warehouse. Adjustments, transfers, receipts, sales and returns create movement history. Optional batch, lot and expiry details can be recorded. The available quantity is protected from becoming negative.

## 5. Demand intelligence

The system rebuilds daily sales history by SKU, warehouse and UTC business date from sales and return movements. For SKUs with enough recent observations, the current Python model uses a recursive lag-7 baseline: it repeats the recent weekly pattern across the requested forecast horizon (currently 14 days). The application also retains deterministic exponential smoothing for explicitly selected SKU requests.

Anomaly checks compare activity with a rolling statistical baseline. Reorder suggestions use lead-time demand, safety stock, available stock, minimum order quantity (MOQ) and pack size. Each recommendation stores its inputs and formula version so a manager can understand and reproduce the calculation. Recommendation approval creates a draft purchase order; it does not directly change stock or automatically submit the order.

Insufficient history is surfaced as a status instead of being presented as a zero-demand forecast. The training CSV and other offline model artifacts do not represent current SKU-by-warehouse sales and are not used as a substitute for matching operational history.

## 6. Security, reliability and dashboards

Access uses short-lived JWT access tokens and rotated HTTP-only refresh tokens. Protected actions check the user's role and resource scope, such as assigned warehouses or supplier account. Passwords are hashed. Server-side validation, secure headers, CORS restrictions, rate limiting, structured logs and health checks support safer operation.

Critical actions such as inventory changes, order decisions, payment state changes, purchase-order decisions and recommendation approvals are audit logged. Manager dashboards show stock value, low stock, open purchase orders, alerts, sales trends and pending recommendations. Admin and supplier dashboards show metrics scoped to each role.

# 7. Technology and current state

| Area | Technology |
|---|---|
| Frontend | React, Vite, JavaScript/JSX, HTML and CSS |
| Backend | Node.js and Express.js modular monolith |
| Database | PostgreSQL |
| ORM | Prisma |
| Intelligence | Node.js deterministic formulas; Python lag-7 predictor |
| Deployment | Docker services and reverse proxy |
| Payments | Mock provider behind a payment-provider interface |

The project documents core local workflows as implemented. Real payment provider selection and credentials remain to be configured. The current API Docker image does not include the Python runtime and ML files, so forecasting requires the native development setup. Kafka, microservices, automated purchase-order approval and delivery-route optimization are outside the initial release scope.

# 8. Suggested classroom demo

1. Sign in as a warehouse manager and open the dashboard.
2. Review low stock and a demand/reorder recommendation.
3. Create a purchase order for a supplier.
4. Switch to the supplier account and accept the assigned order; record shipment details.
5. Return as manager and receive goods, optionally entering batch information.
6. Show the updated inventory, order status and audit history.

### Short closing statement

SupplySense connects the path from customer demand to warehouse replenishment. Its main contribution is one auditable workflow that combines commerce, inventory, suppliers and explainable reorder decisions while protecting stock consistency.

Project sources used: README.md; docs/requirements.md; docs/architecture.md; docs/phase-9-supply-chain.md; docs/phase-11-intelligence.md; docs/phase-12-dashboards.md.